```
<div class="modal-footer">
    <button type="button" class="btn btn-secondary"
     ➥ data-bs-dismiss="modal">Close</button>
    <button type="button" class="btn btn-success"
     ➥ data-bs-dismiss="modal">Book</button>
</div>
```

Finally, you need to wire the new button up to a click handler that posts the form to the page handler method. The jQuery version is shown in the following listing.

Listing 11.12 Adding a new handler to the new button using jQuery

```
$('.btn-success').on('click', function(){
    const data = $('#booking-form').serialize();                ❶
    $.post('?handler=booking', data, function(totalCost){       ❷
        alert(`Your stay will cost ${totalCost}`);              ❸
    });
});
```

❶ Use the jQuery serialize method to convert the form field values into a URL-encoded string for posting.

❷ Post to the page handler and define a callback function that takes the response as a parameter.

❸ Incorporate the response into a string that is displayed in a browser alert.

You are ready to test this out. Once you have the new version of the City page in your browser, make sure you have your developer tools open on the network tab. Then click a property, enter a start and an end date for the booking, and then specify the number of guests. Click the Book button. You should now see a 400 error code in the Network tab (figure 11.4).

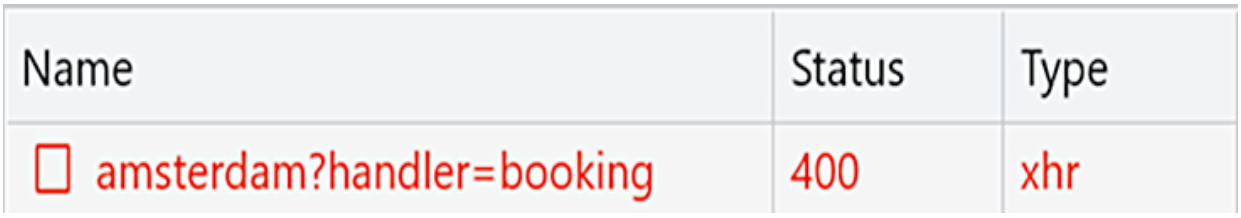


Figure 11.4 The request fails with a 400 error code.